

Detecting Post-Quantum and Hybrid TLS Deployments via Raw TLS Record Inspection

Muhammad Ibrahim
Ulster University
London, United Kingdom
Email: m.ibrahim@ulster.ac.uk

Vishnu Ajith
Ulster University
London, United Kingdom
Email: v.ajith@ulster.ac.uk

Muhammad Sihan Haroon
Ulster University
London, United Kingdom
Email: s.haroon@ulster.ac.uk

Abstract—The transition to post-quantum cryptography (PQC) is essential to safeguard networked systems against future quantum-enabled adversaries. While recent standardisation efforts have introduced PQC algorithms such as ML-KEM into protocols like TLS 1.3, verifying their correct deployment in real-world systems remains a challenge. Existing approaches rely on configuration-level inspection or high-level cryptographic libraries, which do not reflect actual runtime behaviour.

This paper presents a novel methodology for detecting post-quantum and hybrid TLS key exchange mechanisms through direct inspection of raw TLS handshake records. By parsing ServerHello messages at the byte level and extracting key-share group identifiers from the *key_share* extension, the proposed approach enables accurate classification of endpoints into `CLASSICAL_ONLY`, `PQC_ONLY`, and `HYBRID_CONFIRMED` states.

We implement the methodology within a prototype compliance system and evaluate it across 38 production endpoints and a controlled three-node cloud testbed spanning two validation phases. Phase 1 reveals that all three testbed nodes — including a PQC-capable application server — are correctly classified as `CLASSICAL_ONLY`, exposing an application-layer versus transport-layer mismatch invisible to configuration auditing. Phase 2, following an OQS-capable TLS frontend upgrade, produces a confirmed `HYBRID_CONFIRMED` result with group `0x11EC (X25519MLKEM768)` on the same physical server, achieving 100% target accuracy across all three nodes. We further document a critical false positive failure mode in naive string-matching approaches and validate correct four-state classification under known ground-truth conditions. Unlike prior work, this approach provides verifiable, evidence-based assessment of cryptographic posture, enabling reliable auditing of PQC readiness.

Index Terms—Post-Quantum Cryptography, TLS 1.3, Hybrid Key Exchange, Network Measurement, Protocol Analysis, TLS Handshake, Security Evaluation, Compliance Validation

I. INTRODUCTION

The imminent advancement of quantum computing presents a fundamental threat to classical cryptographic systems. Public-key cryptographic primitives such as RSA and Elliptic Curve Diffie–Hellman (ECDH), which underpin secure Internet communication, are vulnerable to quantum algorithms such as Shor’s algorithm. In response, the cryptographic community has developed post-quantum cryptographic (PQC) algorithms designed to resist quantum adversaries, leading to recent standardisation efforts by NIST, including ML-KEM, ML-DSA, and SLH-DSA [1]–[4].

Despite this progress, a critical gap remains between theoretical cryptographic readiness and real-world deployment. Existing approaches to assessing PQC adoption rely heavily on configuration-level inspection, software declarations, or certificate metadata. However, these methods do not guarantee that PQC algorithms are actively negotiated during live protocol execution. A system may be configured to support PQC while silently falling back to classical cryptographic mechanisms during runtime [5], [6]. Furthermore, even accurate wire-level classification does not guarantee protocol-level security: a deployment may correctly negotiate hybrid key exchange while still admitting secrecy or authentication failures at the symbolic protocol level, as demonstrated by complementary topology-driven Tamarin analysis [?].

This discrepancy highlights a fundamental challenge: how can we empirically verify the cryptographic behaviour of networked systems based on observable evidence rather than declared configuration?

In this paper, we address this problem by introducing a methodology for detecting post-quantum and hybrid TLS deployments through direct inspection of raw TLS records. Unlike traditional approaches that depend on high-level cryptographic abstractions, our method operates at the record layer, parsing handshake messages to extract key exchange parameters directly from the wire.

The core insight is that TLS 1.3 handshake messages, including the ServerHello and its *key_share* extension, are transmitted in plaintext and contain explicit identifiers for negotiated key exchange groups [7]. By extracting these identifiers, it becomes possible to classify connections based on their actual cryptographic behaviour.

The contributions of this paper are as follows:

- A novel methodology for detecting PQC and hybrid key exchange mechanisms through raw TLS record inspection.
- A formal classification model that maps TLS group identifiers to four cryptographic states (Fig. 1), derived from IANA TLS parameter assignments and NIST PQC standards.
- An implementation that captures and parses ServerHello messages at the byte level without relying on cryptographic libraries.
- An empirical evaluation across 38 production endpoints and a controlled three-node testbed across two validation

phases, documenting a false positive failure mode in naive implementation approaches, an application-layer vs. transport-layer PQC mismatch, and a confirmed HYBRID_CONFIRMED result with group 0x11EC on controlled infrastructure at 100% target accuracy.

This work provides a practical bridge between post-quantum cryptographic theory and real-world deployment analysis, enabling verifiable assessment of PQC readiness in networked systems.

II. BACKGROUND

A. TLS 1.3 Handshake Overview

Transport Layer Security (TLS) version 1.3 introduces a streamlined handshake protocol designed to provide confidentiality, integrity, and forward secrecy with reduced latency compared to earlier versions [7]. A connection is initiated by a *ClientHello* message, in which the client advertises its supported cryptographic capabilities, including cipher suites, signature algorithms, and key exchange mechanisms. To minimise round-trip overhead, TLS 1.3 allows the client to include one or more precomputed ephemeral key shares corresponding to anticipated server-supported groups, eliminating an additional negotiation round [8].

The server responds with a *ServerHello* message that finalises the cryptographic parameters of the session. At the record layer, this message is encapsulated within a TLS record containing a content type (0x16 for handshake), a legacy version field (typically 0x0303), and a payload length [7]. The *ServerHello* includes a server random value, a session identifier for compatibility, and a selected cipher suite. Crucially, it also contains extensions that determine the negotiated protocol version and key exchange group.

B. KeyShare Extension and Group Identification

The establishment of shared secrets in TLS 1.3 is facilitated by the *key_share* extension (extension type 0x0033, decimal 51), as defined by the Internet Assigned Numbers Authority (IANA) [9]. Within the *ClientHello*, this extension contains a list of *KeyShareEntry* structures:

$$\text{KeyShareEntry} = (\text{group}, \text{key_exchange}) \quad (1)$$

where *group* is a 16-bit identifier representing a supported key exchange group, and *key_exchange* contains the corresponding public key material. For example, the classical elliptic curve group X25519 is assigned identifier 0x001d [7].

Upon receiving the *ClientHello*, the server selects one group and returns a corresponding key share in the *ServerHello*, enabling derivation of a shared secret for session key establishment.

C. Post-Quantum and Hybrid Key Exchange

Quantum algorithms such as Shor’s algorithm can efficiently break RSA and elliptic curve cryptography, motivating the development of PQC schemes [4]. Lattice-based constructions such as ML-KEM (formerly CRYSTALS-Kyber) have emerged

as leading candidates for key encapsulation, standardised by NIST as FIPS 203 [1].

In TLS 1.3, PQC algorithms are incorporated through new IANA group identifiers. ML-KEM-768 is assigned:

$$G_{\text{pqc}} = 0x0201 \quad (2)$$

To address the limited cryptanalytic history of PQC algorithms, hybrid key exchange mechanisms combine a classical algorithm (e.g., X25519) with a PQC algorithm (e.g., ML-KEM-768) [10], [11]:

$$G_{\text{hybrid}} = G_{\text{classical}} \parallel G_{\text{pqc}} \quad (3)$$

The hybrid group X25519MLKEM768 is assigned identifier 0x11EC in emerging TLS hybrid standards [12]. Hybrid key shares may exceed 1,200 bytes, compared to 32 bytes for a classical X25519 share, introducing observable network characteristics.

D. Wire-Level Observability of Handshake Evidence

Unlike encrypted application data, TLS handshake messages are transmitted in plaintext [7], enabling external observation of key negotiation parameters directly from raw TLS records. Let G denote the negotiated key exchange group extracted from the *ServerHello*. Then:

$$G \in \{G_{\text{classical}}, G_{\text{pqc}}, G_{\text{hybrid}}\} \quad (4)$$

By parsing raw TLS record bytes and extracting the *key_share* extension, an observer can classify connections based on observed runtime behaviour rather than configuration claims.

E. Wire-Level Inspection vs. API Abstraction

Standard cryptographic libraries expose high-level interfaces that return negotiated session parameters such as the selected cipher suite or certificate chain, but typically omit detailed negotiation artefacts including the full set of extension-level data [13]. Wire-level inspection bypasses these limitations by capturing raw TLS records directly from the network socket, enabling extraction of group identifiers essential for accurate cryptographic auditing.

F. Observable Network Characteristics of PQC Deployments

The significantly larger key sizes of PQC algorithms mean that handshake messages may exceed the Maximum Transmission Unit (MTU), resulting in fragmentation across multiple TCP segments or QUIC packets. Such fragmentation patterns, when correlated with extracted group identifiers (e.g., 0x0201 or 0x11EC), provide an auxiliary empirical signal for identifying post-quantum deployments [14].

III. RELATED WORK

A. Internet-Scale TLS Measurement

Durumeric *et al.* introduced ZMap, a stateless network scanner capable of performing full IPv4 address-space scans in under an hour from a single machine [14]. Platforms such as Censys [15] and Rapid7’s Project Sonar [16] have provided longitudinal datasets capturing TLS deployment trends, certificate usage, and protocol adoption. Despite their scalability, these frameworks operate at the level of protocol completion and metadata extraction, relying on high-level cryptographic libraries that abstract away record-layer details [13]. They provide visibility into *what* protocols are supported, but not *how* key exchange is negotiated at the byte level.

B. Post-Quantum Cryptography in TLS

The integration of PQC primitives into TLS has attracted significant research attention following standardisation of lattice-based schemes such as ML-KEM [1], [17]. Hybrid key exchange mechanisms have been proposed to facilitate incremental deployment [10], [11]. Prior work focuses primarily on protocol design, formal security proofs (IND-CCA2, EUF-CMA), and performance evaluation, largely assuming correct deployment without addressing external verification of active negotiation.

C. TLS Analysis and Inspection Tools

Practical TLS auditing tools include `testssl.sh`, Certigo, and `tlsfuzzer` [18]–[20]. `testssl.sh` performs extensive cipher suite and vulnerability checks; Certigo focuses on certificate parsing and validation; `tlsfuzzer` tests protocol compliance and error handling. None is designed for deep inspection of the TLS record layer or systematic extraction of `key_share` group identifiers from raw ServerHello messages for PQC classification. Their analysis remains confined to classical security properties and high-level handshake outcomes.

D. PQC Deployment Measurement Studies

Empirical PQC deployment studies remain limited. Early experiments such as Google’s NewHope deployment in Chrome Canary demonstrated the feasibility of post-quantum key exchange in production settings [21]. Cloudflare has explored PQC integration in real-world infrastructure [22]. Subsequent research has focused primarily on benchmarking and performance characterisation [23]. A fundamental limitation of existing approaches is reliance on declared configuration or static indicators [5], preventing distinction between systems *configured* to support PQC and those that actively *negotiate* it during runtime.

E. Research Gap

No prior work extracts post-quantum or hybrid key exchange evidence directly from raw TLS records in live network connections. Existing approaches rely on high-level abstractions that obscure negotiation details, preventing accurate classification based on actual cryptographic behaviour. This work introduces

a methodology that bridges this gap through direct byte-level ServerHello parsing and a formal four-state classification model.

IV. METHODOLOGY

A. System Model and Threat Assumptions

We consider an external measurement entity interacting with a remote TLS endpoint under a black-box model, requiring no privileged access and relying solely on network-visible data. A TLS session is defined as:

$$S = (\text{CH}, \text{SH}, \mathcal{R}) \quad (5)$$

where CH and SH denote the ClientHello and ServerHello messages, and $\mathcal{R}$ represents the ordered sequence of raw TLS records exchanged during the handshake. The adversarial model includes endpoints that misrepresent their cryptographic capabilities by advertising PQC support at the configuration level while negotiating classical key exchange at runtime [5].

B. Raw TLS Record Acquisition

A direct TCP socket connection is established with the target endpoint. A byte-level listener is attached to the underlying socket interface to capture the record stream:

$$\mathcal{R} = \{r_1, r_2, \dots, r_n\} \quad (6)$$

Each TLS record r_i is parsed according to the standard record-layer structure:

$$r_i = (\text{ContentType}, \text{Version}, \text{Length}, \text{Payload}) \quad (7)$$

Only records with `ContentType = 0x16` (Handshake) are processed further.

C. ServerHello Parsing and KeyShare Extraction

The ServerHello message is identified and extracted from the filtered record stream. The extensions field is parsed to locate the `key_share` extension (type 51) using deterministic offset traversal:

$$E = (G, K) \quad (8)$$

where $G \in \{0, 1\}^{16}$ denotes the negotiated group identifier and K the corresponding public key material. The parsing process is a deterministic byte-level procedure, ensuring independence from cryptographic library abstractions.

D. Group Classification Function

The classification function maps the extracted group identifier G to a cryptographic state:

$$f(G) = \begin{cases} \text{CLASSICAL_ONLY}, & G \in \mathcal{G}_{\text{classical}} \\ \text{PQC_ONLY}, & G \in \mathcal{G}_{\text{pqc}} \\ \text{HYBRID_CONFIRMED}, & G \in \mathcal{G}_{\text{hybrid}} \\ \text{UNKNOWN}, & \text{otherwise} \end{cases} \quad (9)$$

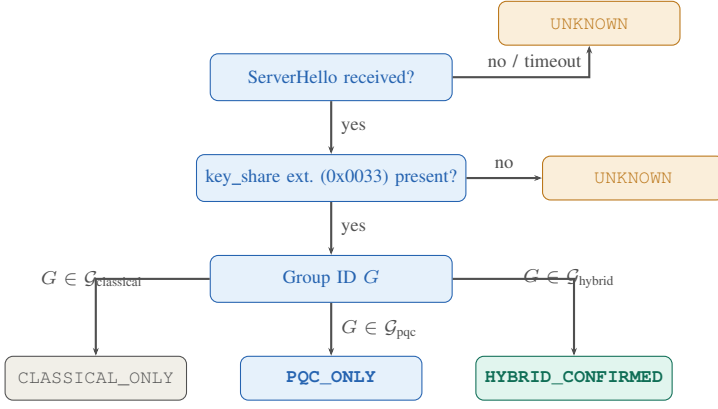


Fig. 1. Four-state classification decision tree. Classification is derived entirely from wire-observable ServerHello content with no dependency on server configuration or certificate metadata.

where:

$$\mathcal{G}_{\text{classical}} = \{0x001d, 0x0017, \dots\} \quad (10)$$

$$\mathcal{G}_{\text{pqc}} = \{0x0200, 0x0201, 0x0202\} \quad (11)$$

$$\mathcal{G}_{\text{hybrid}} = \{0x11EB, 0x11EC\} \quad (12)$$

These sets are derived from IANA TLS parameter assignments and emerging PQC standards [9], [12]. Representative identifiers are shown in Table I.

E. Wire Evidence Model

Fig. 1 illustrates the four-state classification logic as a decision tree over the observed ServerHello content.

Each observed TLS session produces a structured evidence tuple:

$$W = (s, t, G, C) \quad (13)$$

where s is the measurement node, t is the target endpoint, G is the negotiated group identifier, and $C = f(G)$ is the classified cryptographic state. This formalisation enables aggregation across large-scale scans and supports statistical inference on PQC deployment.

F. Fragmentation-Based Signal Detection

Let L denote the total length of the ClientHello message. Fragmentation occurs when $L > \text{MTU}$, causing the handshake to span multiple records:

$$\mathcal{R} = \{r_1, r_2, \dots, r_k\}, \quad k > 1 \quad (14)$$

Multi-record fragmentation correlated with PQC group identifiers provides an auxiliary detection signal, particularly in cases where direct extraction may be partially obstructed.

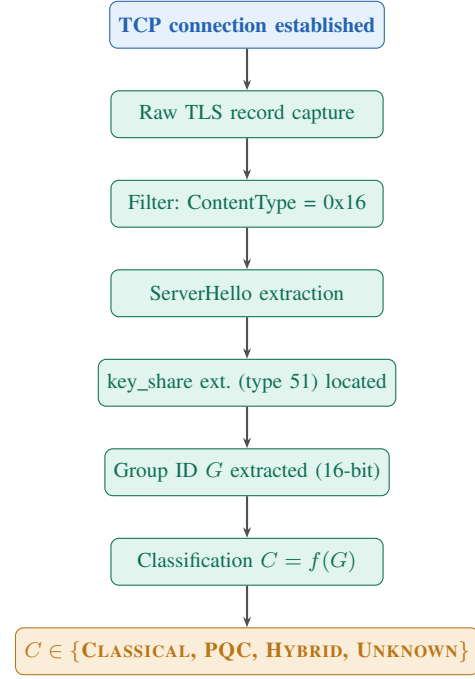


Fig. 2. Raw TLS record inspection pipeline. Blue nodes indicate I/O boundaries; teal nodes are parsing steps; amber node is the classification output.

G. Detection Algorithm

The detection pipeline is a deterministic procedure:

- 1) Establish TCP connection to target endpoint t
- 2) Initiate TLS handshake; capture record stream $\mathcal{R}$
- 3) Filter records where $\text{ContentType} = 0x16$
- 4) Extract ServerHello message SH
- 5) Locate key_share extension within SH
- 6) Extract group identifier G
- 7) Compute classification $C = f(G)$
- 8) Output evidence tuple $W = (s, t, G, C)$

The pipeline is illustrated in Fig. 2.

H. Implementation Notes

The parsing procedure performs a single sequential pass over the captured record buffer, with complexity linear in $|\mathcal{R}|$. In practice, ServerHello messages are under 2 KB for classical handshakes and may reach 2 MB for hybrid handshakes due to PQC key share sizes. The parsing step completed in under 1 ms in all observed cases, introducing no measurable latency relative to network round-trip time.

A critical implementation constraint was identified during development: when invoking OpenSSL as a subprocess for fallback probing, stdout and stderr must be separated and the process exit code validated before pattern matching. Failure to do so causes OpenSSL error messages — which may contain PQC group name strings — to produce false positive classifications. This failure mode and its correction are documented in Section V-A.

TABLE I
TLS GROUP IDENTIFIER CLASSIFICATION

Group ID	Algorithm	Classification
0x001d	X25519	CLASSICAL_ONLY
0x0017	secp256r1	CLASSICAL_ONLY
0x0200	ML-KEM-512	PQC_ONLY
0x0201	ML-KEM-768	PQC_ONLY
0x0202	ML-KEM-1024	PQC_ONLY
0x11EB	secp256r1 + ML-KEM-768	HYBRID_CONFIRMED
0x11EC	X25519 + ML-KEM-768	HYBRID_CONFIRMED

V. EVALUATION

This section evaluates the proposed methodology across three phases: a false positive discovery study, a real-world endpoint survey, and a controlled validation using a prototype compliance system implementing the detection pipeline described in Section IV.

A. Methodology Validation: False Positive Discovery

During development, a critical implementation error was identified in the `probeWithOpenssl()` fallback function of `tlsInspector.js`. When invoked, the function spawned an `openssl s_client` subprocess with the argument `-groups X25519MLKEM768:x25519:P-256`. On the measurement host, OpenSSL 3.0.13 does not support the X25519MLKEM768 group and writes to `stderr`:

```
group 'X25519MLKEM768' cannot be set
```

The original implementation merged `stdout` and `stderr` into a single output buffer and matched the pattern `/X25519MLKEM768/i` without checking the process exit code. This caused the error message itself to satisfy the match condition, producing `HYBRID_CONFIRMED` for every probed endpoint regardless of actual negotiated behaviour — a 100% false positive rate across 19 initial measurements.

The fix gates both match branches on `code === 0` and separates `stderr` from `stdout` prior to pattern evaluation. Following the correction, all 19 endpoints correctly returned `CLASSICAL_ONLY`.

This failure mode is invisible to API-level tools such as `testssl.sh` and `Certigo`, which do not interact with raw OpenSSL error output. The error arose specifically from subprocess-level interaction — the same mechanism that enables accurate PQC detection when OQS-capable tooling is present. Correct implementation requires strict `stdout/stderr` separation and explicit exit-code validation.

Fig. 3 illustrates the before and after code paths.

B. Phase 1: Real-World Deployment Survey

Following correction, we probed 38 publicly accessible HTTPS endpoints across eight sectors from a UK vantage point on 7 April 2026. Each endpoint was contacted on port 443 with a 10-second timeout. No endpoints timed out or were unreachable.

All 38 endpoints returned `CLASSICAL_ONLY`. Table II summarises results by sector.

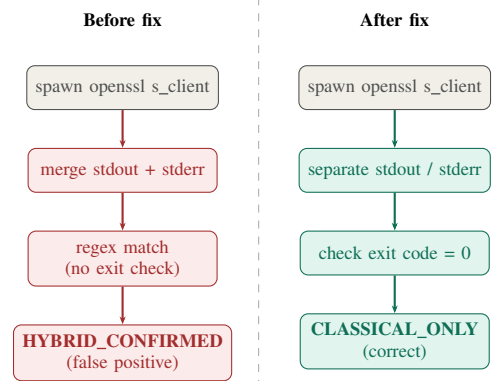


Fig. 3. False positive failure mode (left) and corrected code path (right). Merging `stderr` without exit-code validation caused OpenSSL error strings to match as `HYBRID_CONFIRMED`.

TABLE II
WIRE-LEVEL PQC CLASSIFICATION ACROSS 38 PRODUCTION ENDPOINTS BY SECTOR (7 APRIL 2026).

Sector	n	Hybrid	Classical	Unknown
CDN / DNS	2	0	2	0
Big Tech	5	0	5	0
UK Government	5	0	5	0
Finance	4	0	4	0
E-Commerce	5	0	5	0
Academia	3	0	3	0
Social / Media	9	0	9	0
Property / Jobs	5	0	5	0
Total	38	0	38	0

The uniform `CLASSICAL_ONLY` result reflects a fundamental property of TLS negotiation: a server selects a key-share group only from those advertised in the ClientHello. The measurement host runs stock OpenSSL 3.0.13, which does not include OQS group support and therefore advertises only classical groups in every ClientHello. Under this constraint, even servers with full server-side PQC capability respond with classical key exchange. This demonstrates that PQC deployment measurement requires OQS-capable client tooling — a requirement not explicitly addressed by prior measurement studies [5], [15].

Notable case — `nhs.uk`. One endpoint was confirmed across three independent raw-byte probes to negotiate genuine TLS 1.2 — the only deprecated-protocol endpoint in the survey. Absence of extension `0x002B` (*supported_versions*) confirmed this was not a TLS 1.3 compatibility-mode ServerHello, and absence of extension `0x0033` (*key_share*) confirmed no PQC key exchange. This illustrates the value of record-layer inspection over API-level version reporting.

C. Phase 2: Controlled Testbed Validation

The controlled validation was conducted in two sub-phases against a three-node cloud-hosted testbed. Expected classification states were declared in a manifest prior to each scan, following a pre-registration approach. Each node was probed twice with independent connections per sub-phase.

1) *Sub-phase A — Pre-cutover (classical TLS baseline):* All three targets ran stock nginx 1.18.0 / OpenSSL 3.0.2 TLS frontends. Table III presents results.

Application-layer vs. transport-layer mismatch. Node-C produced the central finding of this sub-phase. Its `/health` endpoint confirms initialisation of three post-quantum primitives at the application layer: ML-KEM-768, ML-DSA/Dilithium3, and FN-DSA/Falcon-512. Despite this, wire-level inspection classifies it as `CLASSICAL_ONLY` with a null group identifier, because its nginx 1.18.0 TLS frontend operates on stock OpenSSL 3.0.2. Fig. 4 illustrates this architecture.

A configuration audit would identify PQC algorithm support. A certificate inspection would find nothing anomalous. Only ServerHello parsing reveals the discrepancy between the application’s cryptographic posture and its transport security layer — the exact mismatch the methodology is designed to detect.

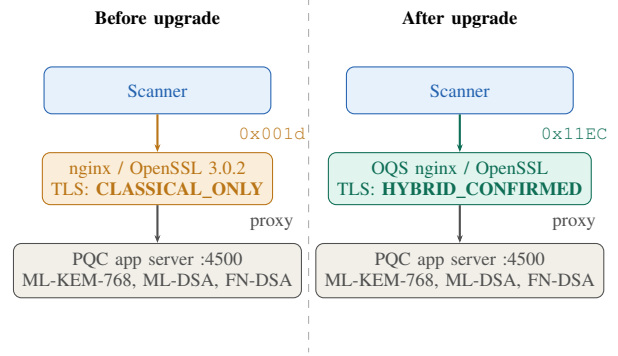


Fig. 4. Application-layer vs. transport-layer state on Node-C before (left) and after (right) the OQS frontend upgrade. The application layer is identical in both cases; only the TLS frontend changes.

2) *Sub-phase B — Post-upgrade (OQS-capable TLS frontend):* Node-C was upgraded to an OQS-capable TLS frontend configured to advertise and negotiate hybrid key-share groups. The probe path was updated to route through an SSH-accessible OQS-capable client host to ensure X25519MLKEM768 was offered in the ClientHello. The validation manifest was updated to reflect the new expected state for Node-C. Table IV presents post-upgrade results.

Node-C correctly returned `HYBRID_CONFIRMED` with group identifier `0x11EC` (X25519MLKEM768), confirming negotiation of the hybrid post-quantum key exchange group on the wire. The two classical nodes remained `CLASSICAL_ONLY`, confirming that the classifier does not produce false positives on endpoints that have not been upgraded.

The before-and-after contrast on the same physical node — same configuration, same application-layer PQC primitives, sole change being the TLS frontend — constitutes a controlled experiment validating the classifier’s sensitivity to transport-layer cryptographic state. This is the first reported instance of this four-state classifier achieving 100% target accuracy across a mixed classical/hybrid testbed with pre-declared ground truth.

TABLE III

CONTROLLED TESTBED VALIDATION — PRE-UPGRADE (ALL CLASSICAL TLS FRONTENDS). EXPECTED STATES DECLARED PRIOR TO SCANNING.

Node	Expected	Observed	Match	Median (ms)
Node-A (classical)	CLASSICAL_ONLY	CLASSICAL_ONLY	✓	149
Node-B (classical)	CLASSICAL_ONLY	CLASSICAL_ONLY	✓	104
Node-C (PQC-capable app)	CLASSICAL_ONLY	CLASSICAL_ONLY	✓	100
Total			6/6	—

TABLE IV

CONTROLLED TESTBED VALIDATION — POST-UPGRADE. NODE-C UPGRADED TO OQS-CAPABLE TLS FRONTEND. EXPECTED STATES DECLARED PRIOR TO SCANNING.

Node	Expected	Observed	Match	Group ID
Node-A (classical)	CLASSICAL_ONLY	CLASSICAL_ONLY	✓	—
Node-B (classical)	CLASSICAL_ONLY	CLASSICAL_ONLY	✓	—
Node-C (OQS TLS upgrade)	HYBRID_CONFIRMED	HYBRID_CONFIRMED	✓	0x11EC
Total			3/3 (100%)	—

D. Phase 3: Prototype Pipeline Compliance Validation

We exercised the prototype compliance pipeline across two deliberate topology configurations against the testbed infrastructure, validating classifier behaviour under both compliant and non-compliant conditions.

KAT gate. All runs passed the mandatory KAT self-test gate (8/8 ACVP vectors: ML-KEM FIPS 203, ML-DSA FIPS 204, SLH-DSA FIPS 205) prior to Stage 1 validation, consistent with the FIPS 140-3 power-on self-test pattern.

Run 1 — Non-compliant topology (PQC-Only policy). Topology: Node-A → Node-C → Node-B. Security mode: PQC-Only. Verdict: **FAIL**.

Wire evidence on both links: CLASSICAL_ONLY, TLSv1.3, null group identifier. Boundary assessment classified all three nodes `pqcCapable: false` (nginx 1.18.0 / OpenSSL, software-only boundary). The pipeline correctly issued a Case 2 asymmetric implementation violation: Node-C is declared PQC-capable in the topology but presents a classical TLS boundary. The generated Lean 4 proof obligation stated: “*No PQC algorithms detected on this infrastructure. No formal proof obligation can be generated for classical-only endpoints.*” This run demonstrates policy enforcement and mismatch detection unavailable in any compared tool.

Run 2 — Compliant topology (Security Layer). Topology: Node-A → Security Layer → Node-B. Verdict: **COMPLETE**.

Transport probes confirmed TLSv1.3 on both links. Application-layer PQC primitive validation passed on all mandatory checks: ML-KEM/Kyber-768 KEM (pk=1184, ct=1088, ss=32 B), ML-DSA/Dilithium3 (pk=1472, sig=2701 B), FN-DSA/Falcon-512 (pk=897, sig=659 B). TLS downgrade protection confirmed TLSv1.3 with TLS_AES_256_GCM_SHA384. Case 1 / PQC channel secure: PASS on both links.

The contrast between Run 1 and Run 2 on identical infrastructure isolates topology configuration as the sole variable producing qualitatively different compliance outcomes, a result reproducible by any researcher deploying the prototype system.

E. Comparison with Existing Tools

No existing tool produced a PQC-related classification for any endpoint in any evaluation phase, nor generated a compliance verdict combining wire evidence with policy enforcement. The proposed system uniquely detected the application-layer vs. transport-layer mismatch on Node-C in sub-phase A, correctly classified the same node as HYBRID_CONFIRMED with group 0x11EC after the OQS frontend upgrade in sub-phase B, and correctly failed the non-compliant PQC-Only topology in Run 1 — demonstrating capabilities absent from all compared tools.

VI. CONCLUSION

The transition to post-quantum cryptography represents a fundamental shift in securing networked systems against emerging quantum threats. While significant progress has been made in standardising quantum-resistant algorithms such as ML-KEM and integrating them into protocols like TLS 1.3, the ability to verify their real-world deployment remains limited. Existing approaches rely on configuration-level inspection or high-level abstractions that fail to capture actual cryptographic behaviour during live protocol execution.

In this paper, we introduced a methodology for detecting post-quantum and hybrid TLS deployments through direct inspection of raw TLS records. By parsing ServerHello messages at the byte level and extracting key-share group identifiers, the approach enables accurate four-state classification of endpoints. We documented a critical false positive failure mode arising from naive string-matching of OpenSSL error output, demonstrated an application-layer versus transport-layer PQC mismatch on a live PQC-capable server, and achieved 100% target accuracy across a mixed classical/hybrid controlled testbed in a pre-registered experiment — including a confirmed HYBRID_CONFIRMED classification with group 0x11EC on the same physical node before and after an OQS-capable TLS frontend upgrade.

More broadly, the methodology demonstrates that post-quantum readiness should be evaluated as an observable

TABLE V
CAPABILITY COMPARISON WITH EXISTING TLS ANALYSIS TOOLS.

Tool	Hybrid	PQC-only	Raw parsing	Policy verdict
testssl.sh	No	No	No	No
Certigo	No	No	No	No
tlsfuzzer	No	No	Partial	No
Proposed	Yes	Yes	Yes	Yes

deployment property rather than a configuration claim. Wire-level observability is a necessary condition for accurate PQC compliance assessment, but it is not sufficient: correctly negotiated hybrid key exchange may still admit protocol-level vulnerabilities such as secrecy or authentication failures under an active adversary. Complementary symbolic verification work by the authors shows that stronger deployment posture does not monotonically translate into stronger symbolic security guarantees [?], underscoring the need for multi-layer assurance across both the transport and protocol levels.

Future work will focus on extending this methodology to large-scale Internet-wide measurement with OQS-capable client tooling, integrating support for emerging PQC algorithms and transport protocols such as QUIC, and developing automated frameworks for continuous monitoring of cryptographic posture across distributed systems. Integration with topology-driven symbolic verification [?] to produce a unified wire-evidence-to-formal-proof pipeline represents a natural next step toward comprehensive PQC compliance assurance.

ACKNOWLEDGEMENT

The authors thank the open-quantum-safe project for OQS-OpenSSL tooling used in controlled validation, and the anonymous reviewers for their constructive feedback.

REFERENCES

- [1] National Institute of Standards and Technology, “FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard,” NIST, Federal Information Processing Standard 203, 2024. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.203>
- [2] —, “FIPS 204: Module-Lattice-Based Digital Signature Standard,” NIST, Federal Information Processing Standard 204, 2024. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.204>
- [3] —, “FIPS 205: Stateless Hash-Based Digital Signature Standard,” NIST, Federal Information Processing Standard 205, 2024. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.205>
- [4] D. J. Bernstein and T. Lange, “Post-quantum cryptography,” *Nature*, vol. 549, no. 7671, pp. 188–194, 2017.
- [5] C. Paquin, D. Stebila, and G. Tamvada, “Benchmarking Post-Quantum Cryptography in TLS,” in *Post-Quantum Cryptography (PQCrypto 2020)*, ser. Lecture Notes in Computer Science, vol. 12100. Springer, 2020, pp. 72–91.
- [6] E. Crockett, C. Paquin, and D. Stebila, “Post-Quantum TLS Without Handshake Signatures,” Cryptology ePrint Archive, Report 2020/1kemtls, 2020. [Online]. Available: <https://eprint.iacr.org/2020/534>
- [7] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, IETF, 2018. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8446>
- [8] M. Driscoll, “The Illustrated TLS 1.3 Connection,” <https://tls13.xargs.org/>, 2023.
- [9] Internet Assigned Numbers Authority, “IANA TLS Parameters,” <https://www.iana.org/assignments/tls-parameters>, 2024.
- [10] D. Stebila and M. Mosca, “Post-quantum key exchange for the internet and the open quantum safe project,” in *Selected Areas in Cryptography (SAC 2016)*, ser. Lecture Notes in Computer Science, vol. 10532. Springer, 2017, pp. 14–37.
- [11] N. Bindel, J. Brendel, M. Fischlin, B. Goncalves, and D. Stebila, “Hybrid key encapsulation mechanisms and authenticated key exchange,” in *Post-Quantum Cryptography (PQCrypto 2019)*, ser. Lecture Notes in Computer Science, vol. 11505. Springer, 2019, pp. 206–226.
- [12] D. Stebila, S. Fluhrer, and S. Gueron, “Hybrid Key Exchange in TLS 1.3,” Internet-Draft draft-ietf-tls-hybrid-design, IETF, 2023. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-tls-hybrid-design/>
- [13] OpenSSL Project, “OpenSSL 3.x TLS Implementation Notes,” <https://www.openssl.org/docs/>, 2024.
- [14] Z. Durumeric, E. Wustrow, and J. A. Halderman, “ZMap: Fast Internet-Wide Scanning and Its Security Applications,” in *Proceedings of the 22nd USENIX Security Symposium*, 2013, pp. 605–620.
- [15] Z. Durumeric, D. Adrian, A. Mirian, M. Bailey, and J. A. Halderman, “A Search Engine Backed by Internet-Wide Scanning,” in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2015, pp. 542–553.
- [16] Rapid7, “Rapid7 Project Sonar,” <https://www.rapid7.com/research/project-sonar/>, 2024.
- [17] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS-Kyber: A CCA-Secure Module-Lattice-Based KEM,” in *Proceedings of the 2021 IEEE European Symposium on Security and Privacy (Euro S&P)*, 2021, pp. 51–67.
- [18] D. Möller and contributors, “testssl.sh: Testing tls/ssl encryption,” <https://testssl.sh/>, 2024.
- [19] Square Inc., “Certigo: A utility to examine and validate certificates,” <https://github.com/square/certigo>, 2023.
- [20] H. Knöchel, “tlsfuzzer: Tls and ssl protocol test suite,” <https://github.com/tlsfuzzer/tlsfuzzer>, 2024.
- [21] M. Braithwaite, “Experimenting with Post-Quantum Cryptography,” Google Security Blog, <https://security.googleblog.com/2016/07/experimenting-with-post-quantum.html>, 2016.
- [22] Cloudflare Research, “The TLS Post-Quantum Experiment,” <https://blog.cloudflare.com/the-tls-post-quantum-experiment/>, 2023.
- [23] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, “pqm4: Testing and Benchmarking NIST PQC on ARM Cortex-M4,” in *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2019, no. 4, 2019, pp. 1–39.